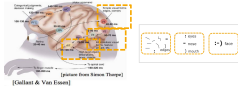


Advanced Artificial Intelligence and Machine Learning

CNN



Learning goals

- CNNs as a locally-connected, weight-sharing alternative to fully-connected networks, suited to spatial and sequential data
- The convolution operation as a learned local feature detector, generalized to 1D, 2D, and 3D data (time series, images, video/volumetric data)
- Padding, stride, and input channels, and how they determine output feature map size
- Sparse connectivity, parameter sharing, and translation equivariance as the properties that make convolutions efficient
- Dilated and transposed convolutions, pooling, and how these ideas culminate in modern architectures (residual/skip connections)

Basic Concepts

CONVOLUTIONAL NEURAL NETWORKS

- CNNs are inspired by the biological visual cortex, where connectivity resembles the mammalian visual system.
- Since their breakthrough win of the 2012 ILSVRC competition, CNNs have become the dominant model class in computer vision.
- Common applications: image classification, object detection/localization, semantic segmentation — plus, outside vision, NLP, audio, and time-series data.
- Basic idea: a CNN automatically extracts spatial (or, more generally, structured) features so a downstream layer can predict optimally.
- Like MLPs, CNNs are built from composable building blocks — but with a different connectivity pattern, covered in this session.

BIOLOGICAL INSPIRATION

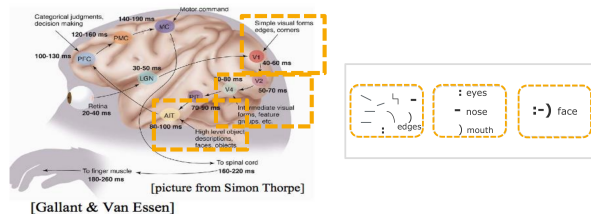


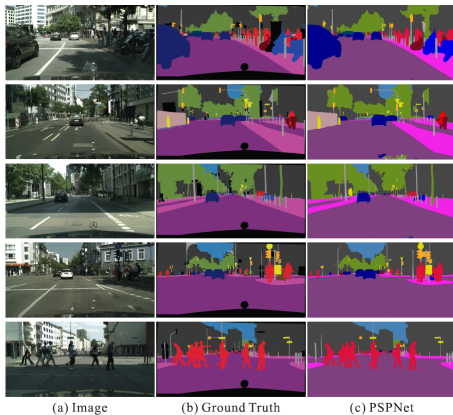
Figure: The ventral (recognition) pathway in the visual cortex has multiple stages — Retina, LGN, V1, V2, V4, PIT, AIT, etc. — consisting of many intermediate representations.

CNNs IN THE REAL WORLD



Figure: Self-driving: a CNN maps raw pixels from a single front-facing camera directly into steering commands, learning to drive in traffic, on marked and unmarked roads, and on highways (The Tesla Team, 2016).

CNNs IN THE REAL WORLD / 2



CNNS IN THE REAL WORLD / 3

Figure: Semantic segmentation: a CNN extracts a feature map, a pyramid-parsing module harvests sub-region representations at multiple scales, and a final convolution produces a per-pixel class prediction (Zhao et al., 2017).

CNNs — A FIRST GLIMPSE

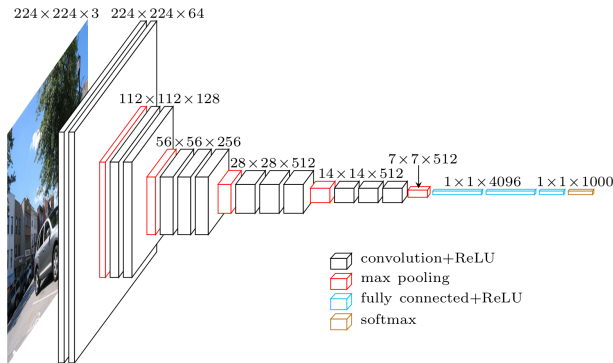


Figure: Example CNN architecture, with typical layer types labeled.

- **Input layer** takes the input data (e.g. image, audio).
- **Convolution layers** extract feature maps from the previous layer.
- **Pooling layers** reduce the dimensionality of feature maps and retain the most salient features.

CNNs — A FIRST GLIMPSE

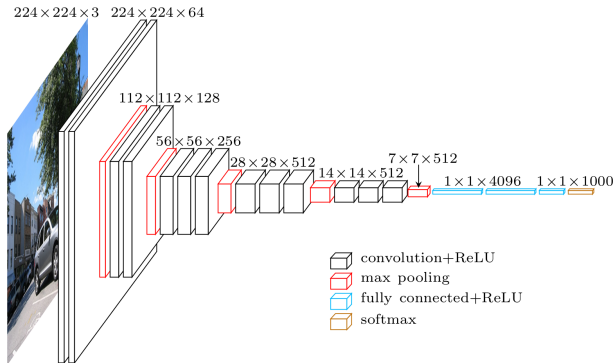


Figure: Example CNN architecture, with typical layer types labeled.

- **Fully connected layers** connect feature-map elements to the output neurons.
- **Softmax** converts output values to class probabilities.

REFERENCES



The Tesla Team. (2016, October 19). *All Tesla cars being produced now have full self-driving hardware: Tesla Switzerland*. Tesla.
https://www.tesla.com/de_ch/blog/all-tesla-cars-being-produced-now-have-full-self-driving-hardware



Zhao, H., Shi, J., Qi, X., Wang, X., & Jia, J. (2017, July). *Pyramid Scene Parsing Network*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

The Convolution Operation

FILTERS TO EXTRACT FEATURES

- Filters have been used in computer vision since the 1970s.
- One prominent example: the **Sobel filter**, which detects edges in images.



FILTERS TO EXTRACT FEATURES / 2

Figure: Sobel-filtered image (Qmegas, 2016).

FILTERS TO EXTRACT FEATURES / 3

- Edges occur where pixel intensity changes rapidly over neighboring pixels, so we approximate the intensity gradient at each pixel.
- Sobel showed that the gradient image $\mathbf{G}_x$ of an original image $\mathbf{A}$ in the x -dimension can be approximated by:

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A} = \mathbf{S}_x * \mathbf{A}$$

where $*$ denotes a **convolution**, not ordinary matrix multiplication.

FILTERS TO EXTRACT FEATURES / 4

- The filter matrix S_x is the product of an **averaging** and a **differentiation** kernel:

$$\underbrace{\begin{bmatrix} 1 & 2 & 1 \end{bmatrix}^T}_{\text{averaging}} \underbrace{\begin{bmatrix} -1 & 0 & +1 \end{bmatrix}}_{\text{differentiation}}$$

FILTERS TO EXTRACT FEATURES / 5

- Similarly, the gradient image $\mathbf{G}_y$ in the y -dimension is approximated by:

$$\mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A} = \mathbf{S}_y * \mathbf{A}$$

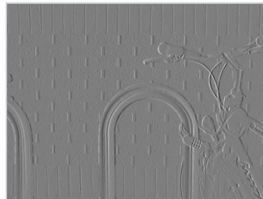
- Combining both gradient images yields a direction-independent gradient magnitude $\mathbf{G}$:

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

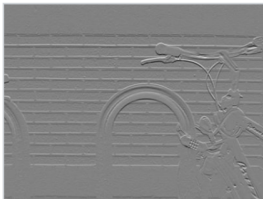
HORIZONTAL VS. VERTICAL EDGES



Input



Vertical edges detected by S_x



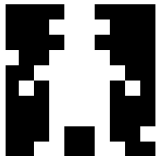
Horizontal edges detected by S_y



Combined

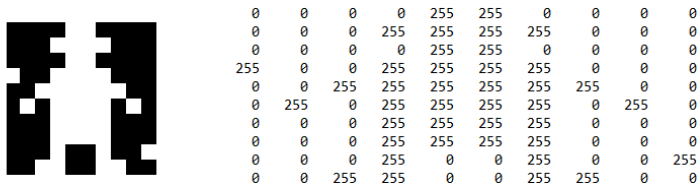
Figure: Sobel-filtered images, normalized per direction (Wikipedia, 2022).

FILTERS TO EXTRACT FEATURES



- Let's apply this to a small dummy image — how do we represent a digital image?

FILTERS TO EXTRACT FEATURES



- Basically as an array of integers.

FILTERS TO EXTRACT FEATURES

Sobel-Operator

$$S_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & \boxed{0} & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

0	0	0	0	255	255	0	0	0	0
0	0	0	255	255	255	255	0	0	0
0	0	0	0	255	255	0	0	0	0
255	0	0	255	255	255	255	0	0	0
0	0	255	255	255	255	255	255	0	0
0	255	0	255	255	255	255	0	255	0
0	0	0	255	255	255	255	0	0	0
0	0	0	255	255	255	255	0	0	0
0	0	0	255	0	0	255	0	0	255
0	0	255	255	0	0	255	255	0	0

- S_x lets us detect vertical edges.

FILTERS TO EXTRACT FEATURES

Sobel-Operator

$$\mathbf{S}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & \boxed{0} & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

0	0	0	0	255	255	0	0	0	0
0	0	0	255	255	255	255	0	0	0
0	0	0	0	255	255	255	0	0	0
255	0	0	255	255	255	255	255	0	0
0	0	255	255	255	255	255	255	0	0
0	255	0	255	255	255	255	0	255	0
0	0	0	255	255	255	255	0	0	0
0	0	0	255	255	255	255	0	0	0
0	0	0	255	0	0	255	0	0	255
0	0	255	255	0	0	255	255	0	0

$$\begin{aligned} (\mathbf{G}_x)_{(i,j)} &= (\mathbf{I} \star \mathbf{S}_x)_{(i,j)} = -1 \cdot 0 + 0 \cdot 255 + 1 \cdot 255 \\ &\quad - 2 \cdot 0 + 0 \cdot 0 + 2 \cdot 255 \\ &\quad - 1 \cdot 0 + 0 \cdot 255 + 1 \cdot 255 \end{aligned}$$

FILTERS TO EXTRACT FEATURES

0	510	1020	510	-510	-1020	-510	0
-255	510	1020	510	-510	-1020	-510	0
-255	765	765	255	-255	-765	-765	-255
255	765	510	0	0	-510	-765	-510
255	510	765	0	0	-765	-510	-255
0	765	1020	0	0	-1020	-765	0
0	1020	765	-255	255	-765	-1020	255
255	1020	0	-765	765	0	-1020	255

- Applying the Sobel operator at every location in the input yields the **feature map**.

FILTERS TO EXTRACT FEATURES



128	191	255	191	64	0	64	128
96	191	255	191	64	0	64	128
96	223	223	159	96	32	32	96
159	223	191	128	128	64	32	64
159	191	223	128	128	32	64	96
128	223	255	128	128	0	32	128
128	255	223	96	159	32	0	159
159	255	128	32	223	128	0	159

- Normalized, the feature map reveals vertical edges. Note the dimensional reduction compared to the input.

WHY DOES THIS MATTER FOR CNNs?

- What we just did was extract a **pre-defined** feature from the input (edges).
- A CNN does almost exactly the same thing: it extracts features from the input — the key difference is that we do not tell it what to look for; **the network learns the filters itself**.
- In a nutshell:
 - Initialize many random filters (like the Sobel filter, but with random entries) and apply them to the input.
 - Feed the resulting feature maps into a classifier (e.g. a feedforward net).
 - Update the filter entries via gradient descent, exactly like any other weight.

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

	a	b	c
	d	e	f
	g	h	i

Input: 3x3x1

w_{11}	w_{12}
w_{21}	w_{22}

Filter: 2x2x1

THE 2D CONVOLUTION

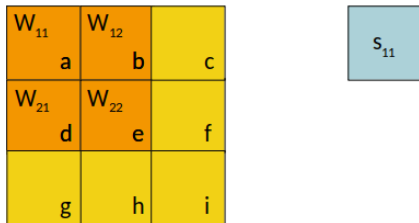
- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

w_{11} a	w_{12} b	c
w_{21} d	w_{22} e	f
g	h	i



THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .



To obtain s_{11} we compute the dot product:

$$s_{11} = a \cdot w_{11} + b \cdot w_{12} + d \cdot w_{21} + e \cdot w_{22}$$

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

	w_{11}	w_{12}
a	b	c
d	e	f
g	h	i

s_{11}	s_{12}
----------	----------

Same for s_{12} : $s_{12} = b \cdot w_{11} + c \cdot w_{12} + e \cdot w_{21} + f \cdot w_{22}$

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

	a	b	c
w_{11}	d	w_{12}	e
w_{21}	g	w_{22}	h

s_{11}	s_{12}
s_{21}	

As well as for s_{21} : $s_{21} = d \cdot w_{11} + e \cdot w_{12} + g \cdot w_{21} + h \cdot w_{22}$

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

a	b	c
d	w_{11} e	w_{12} f
g	w_{21} h	w_{22} i

s_{11}	s_{12}
s_{21}	s_{22}

And finally for s_{22} : $s_{22} = e \cdot w_{11} + f \cdot w_{12} + h \cdot w_{21} + i \cdot w_{22}$

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

a	b	c
d	e	f
g	h	i

Input: 3x3x1

w_{11}	w_{12}
w_{21}	w_{22}

Filter: 2x2x1

s_{11}	s_{12}
s_{21}	s_{22}

Output: 2x2x1

All four entries together give the full 2×2 output feature map.

THE 2D CONVOLUTION

- Suppose we have an input with entries $a, b, \dots, i$ (think: pixel values).
- The filter we apply has weights w_{11}, w_{12}, w_{21} , and w_{22} .

a	b	c
d	e	f
g	h	i

Input: 3x3x1

w_{11}	w_{12}
w_{21}	w_{22}

Filter: 2x2x1

s_{11}	s_{12}
s_{21}	s_{22}

Output: 2x2x1

More generally, let I be the input matrix and W the filter/kernel. Then the output entries are $s_{ij} = \sum_{m,n} I_{i+m-1,j+n-1} w_{mn}$, with m, n ranging over the filter size.

WORKING WITH IMAGES

- Grey-scale images: a matrix of dimension $\text{height} \times \text{width} \times 1$, with pixel values from 0 (black) to 255 (white).
- Color images: a tensor of dimension $\text{height} \times \text{width} \times 3$, where the depth 3 denotes the RGB channels.
- A filter's depth is **always** equal to the input's depth. In practice, filters are usually square, so a single integer defines their spatial size — e.g. a filter of size 2 on a color image has dimensions $2 \times 2 \times 3$.

REFERENCES



Qmegas. (2016). *QMEGAS/Sobel-Operator: PHP implementation of Sobel Operator (Sobel filter)*. GitHub.

<https://github.com/qmegas/sobel-operator>



Wikimedia Foundation. (2022, September 1). *Kantendetektion*. Wikipedia.

<https://de.wikipedia.org/wiki/Kantendetektion>

1D / 2D / 3D Convolutions

1D CONVOLUTIONS

Data situation: sequential, 1-dimensional tensor data.

- Data consists of tensors with shape [depth, xdim].
- Depth 1 (single-channel): univariate time series (e.g. a stock price over time), or functional/curve data.
- Depth > 1 (multi-channel): multivariate time series (e.g. multi-sensor human activity recognition, temperature + humidity in weather forecasting), or text encoded as character-level one-hot vectors.

→ Convolve the data with a 1D kernel.

1D CONVOLUTIONS — OPERATION

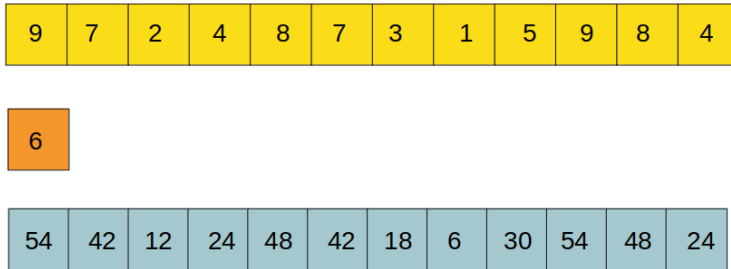


Figure: 1D movement data with depth 1 and filter size 1.

1D CONVOLUTIONS — OPERATION / 2

9	7	2	4	8	7	3	1	5	9	8	4
---	---	---	---	---	---	---	---	---	---	---	---

3	6
---	---

69	33	30	60	66	39	15	33	69	75	48
----	----	----	----	----	----	----	----	----	----	----

Figure: 1D movement data with depth 1 and filter size 2.

1D CONVOLUTIONS — SENSOR AND TIME-SERIES DATA

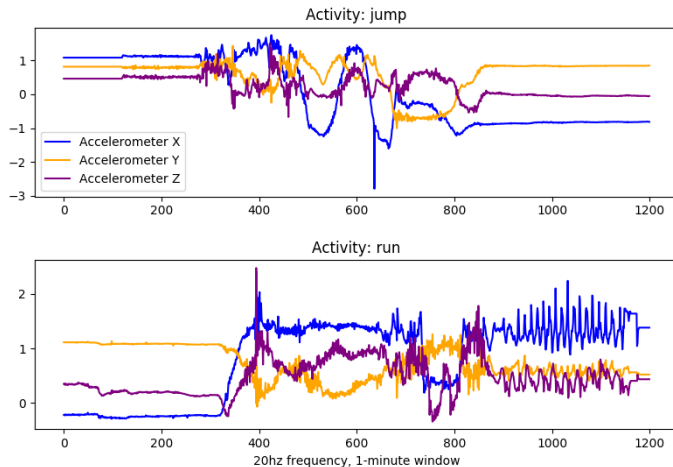


Figure: 1D movement data with depth 3, measured by an accelerometer for human activity recognition

1D CONVOLUTIONS — SENSOR AND TIME-SERIES DATA

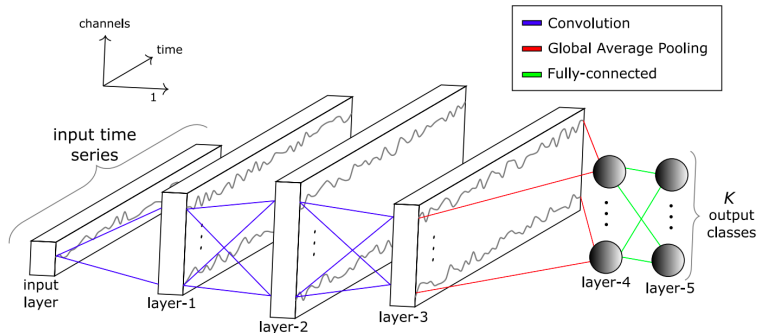


Figure: Time-series classification with a 1D CNN and global average pooling: an input series is convolved by 3 CNN layers, pooled, and fed into a dense layer before the softmax — a classic time-series classification architecture (Ismail Fawaz et al., 2019).

1D CONVOLUTIONS — TEXT MINING

- 1D convolutions also apply to text: e.g. classifying the sentiment of text snippets such as Yelp reviews.



Miriam L.
Munich, Germany

👤 57 friends

★ 437 reviews

📷 450 photos

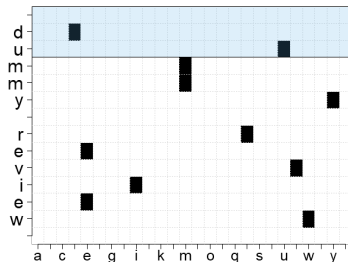


7/18/2010

The LMU is main building one of the most beautiful buildings in München...nicht only in relation to the architecture just great, but above all also if the history here has taken place, is a conscious.

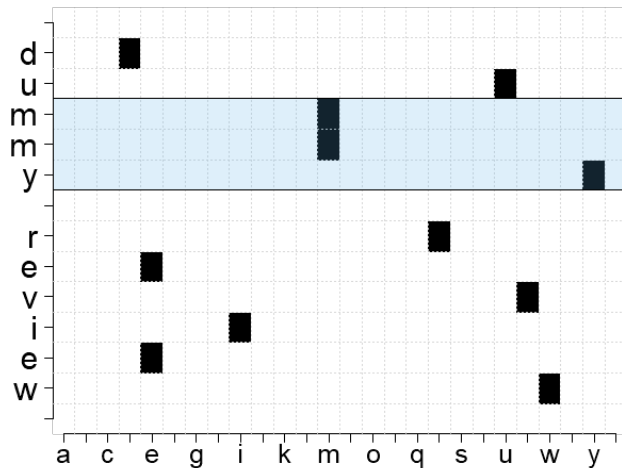
Figure: Sentiment classification: is this a positive review?

1D CONVOLUTIONS — TEXT MINING



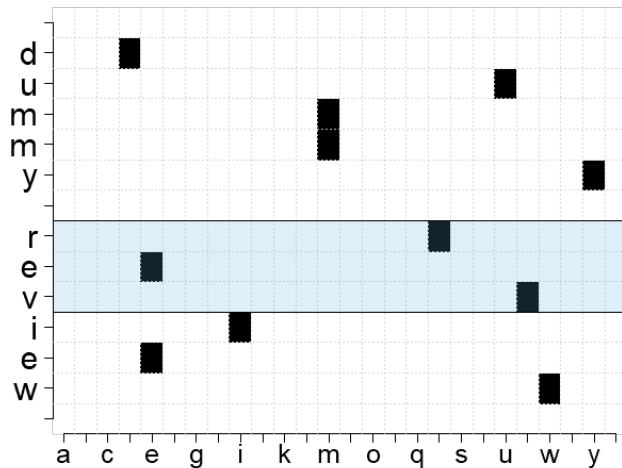
- We encode text on a character level (here: “*dummy review*”). Each character becomes a one-hot vector over the alphabet.
- Reviews are padded or truncated to a fixed length (e.g. 1014 characters).

1D CONVOLUTIONS — TEXT MINING



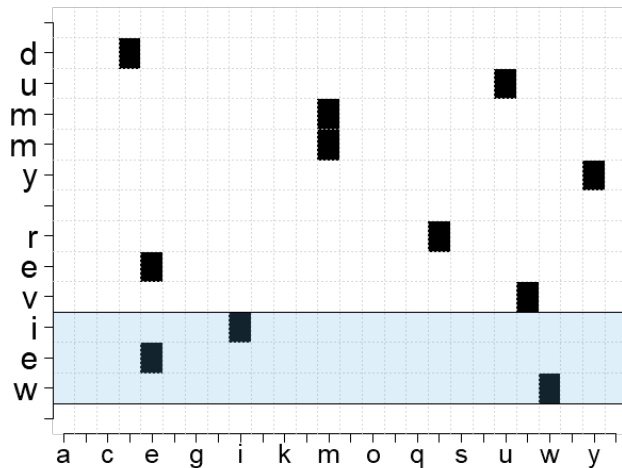
- The result is a 1D signal with *depth* = *alphabet size*.

1D CONVOLUTIONS — TEXT MINING



- The temporal dimension is shown on the y-axis for illustration.

1D CONVOLUTIONS — TEXT MINING



- The 1D kernel (blue) convolves along the temporal dimension, yielding a 1D feature vector.

ADVANTAGES OF 1D CONVOLUTIONS

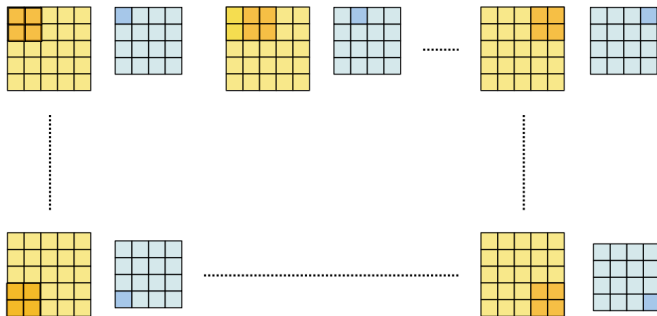
For certain applications, 1D CNNs are preferable to their 2D counterparts:

- **Computational complexity:** forward and backward passes reduce to simple array operations.
- **Training:** shallow 1D architectures are often sufficient for challenging 1D-signal tasks.
- **Hardware:** deep 2D CNNs usually need specialized hardware, while compact 1D CNNs train well on a standard CPU.
- **Application:** low computational cost makes compact 1D CNNs well suited to real-time and low-cost applications, e.g. on mobile or embedded devices.

2D Convolutions

2D CONVOLUTIONS

The basic idea behind a 2D convolution is sliding a small window (the “kernel”/“filter”) over a larger 2D array, computing a dot product between the filter and the corresponding input elements at every position.



2D CONVOLUTIONS / 2

Figure: Applying a 2×2 convolution filter to a 5×5 array, at 16 different positions.

2D CONVOLUTIONS — EXAMPLE

0	1	
3	3	2
2	2	
0	0	1
3	1	2



- In deep learning, convolution means element-wise multiplication followed by addition. For a single-channel image with a 2×2 filter $[[0, 1], [2, 2]]$:

2D CONVOLUTIONS — EXAMPLE

	0	1
3	3	2
	2	2
0	0	1
3	1	2

3	4
---	---

- The filter slides through the input, computing a new feature map.

2D CONVOLUTIONS — EXAMPLE

	3	3	2
0		1	
	0	0	1
2		2	
	3	1	2

3	4
8	

- Notice: stride is 1 and padding is 0 in this example.

2D CONVOLUTIONS — EXAMPLE

	3	3	2
0	0	0	1
3	2	1	2

3	4
8	7

- Each sliding position yields one number; the output here is a 2×2 matrix.

3D Convolutions

3D CONVOLUTIONS

Data situation: 3-dimensional tensor data.

- Data consists of tensors with shape [depth, xdim, ydim, zdim].
- Dimensions can be temporal (e.g. video frames) or spatial (e.g. MRI).
- Examples: human activity recognition in video, disease classification or tumor segmentation on MRI scans.

Solution: move a 3D kernel in x , y , and z direction to capture all relevant information.

3D CONVOLUTIONS — DATA

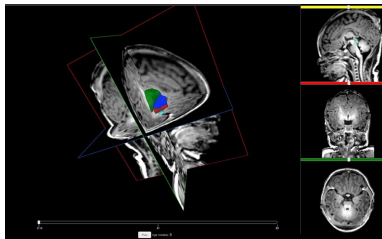


Figure: Depth-1 volumetric data: an MRI scan. Each slice has depth 1, as the frames are black-and-white (Gutman et al., 2014).

3D CONVOLUTIONS — DATA

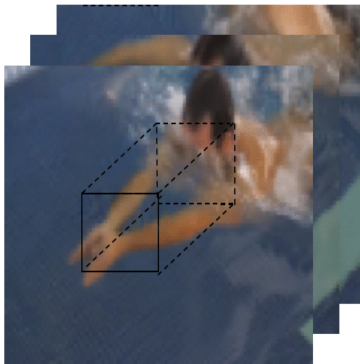


Figure: Volumetric data with depth > 1 : a video snippet for action detection. The video is several RGB (depth-3) slices stacked in temporal order (Ghosh, 2018).

3D CONVOLUTIONS — ARCHITECTURE

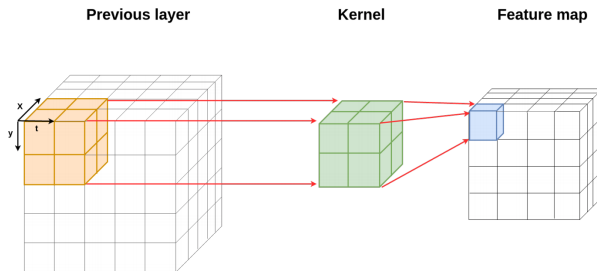


Figure: 3D convolutions yield a 3D output.

3D CONVOLUTIONS — ARCHITECTURE

/ 2

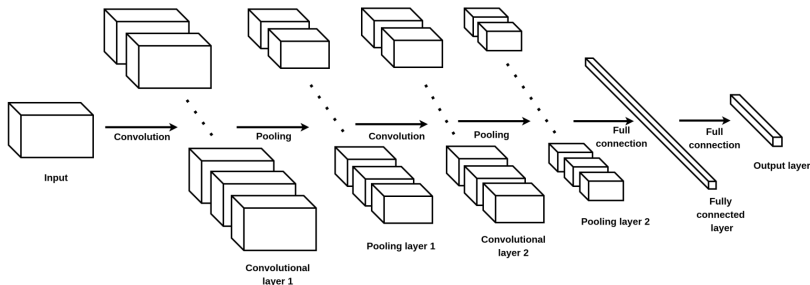


Figure: A basic 3D-CNN architecture: the overall structure is unchanged, but feature maps are now 3D, element-wise activated, and (eventually) pooled in 3 dimensions.

REFERENCES



Ismail Fawaz, H., Forestier, G., Weber, J., Idoumghar, L., & Muller, P.-A. (2019). Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 33.

<https://doi.org/10.1007/s10618-019-00619-1>



Gutman, D., Dunn Jr, W., Cobb, J., Stoner, R., Kalpathy-Cramer, J., & Erickson, B. (2014). Web based tools for visualizing imaging data and development of XNATView, a zero footprint image viewer. *Frontiers in Neuroinformatics*, 8, 53. <https://doi.org/10.3389/fninf.2014.00053>



Ghosh, R. (2018, June 11). *Deep learning for videos: A 2018 guide to action recognition*. <https://blog.qure.ai/notes/deep-learning-for-videos-action-recognition-review>

CNN Components: Channels, Padding, Stride

INPUT CHANNELS

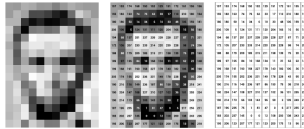


Figure: Left: image of Lincoln; center: pixels labeled with grayscale values; right: grayscale values alone (Levin & Dorsey).

- An image consists of the smallest indivisible segments, called pixels, each with an intensity value.
- A grayscale image has a single input channel, with each pixel's value representing brightness.
- Grayscale values range from 0 to 255 (0 = black, 255 = white).

INPUT CHANNELS — COLOR

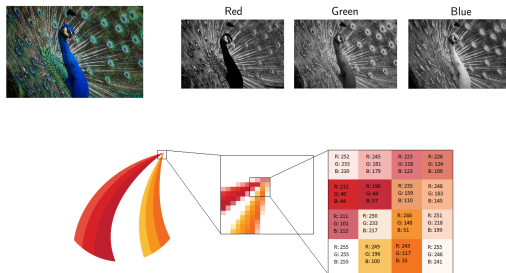
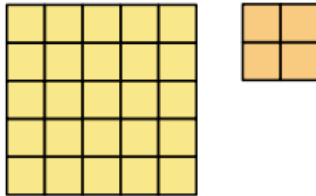


Figure: How a CNN “sees” an image (Maladkar, 2020).

- A color digital image usually has three color channels — Red, Green, Blue (RGB).
- Each pixel is a vector of three numbers (each in $[0, 255]$), one per color channel.

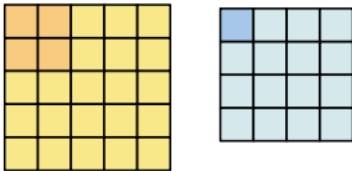
VALID PADDING

Suppose we have an input of size 5×5 and a filter of size 2×2 .



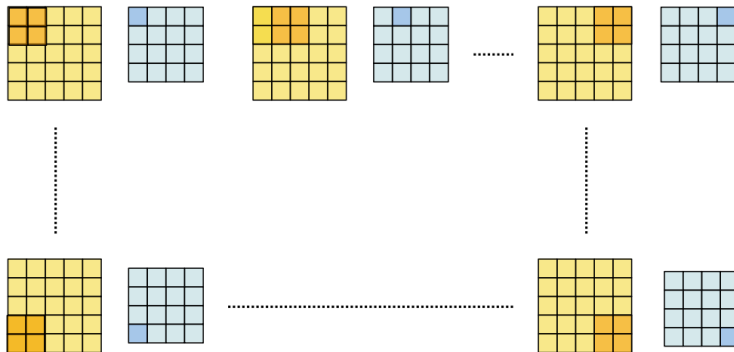
VALID PADDING

The filter is only allowed to move inside the input space.



VALID PADDING

That inevitably reduces the output dimensions.

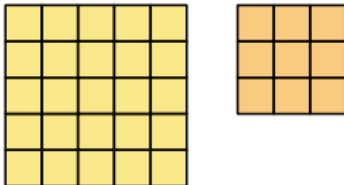


In general, for an input of size $i (\times i)$ and filter size $k (\times k)$, the output feature map size $o (\times o)$ is:

$$o = i - k + 1$$

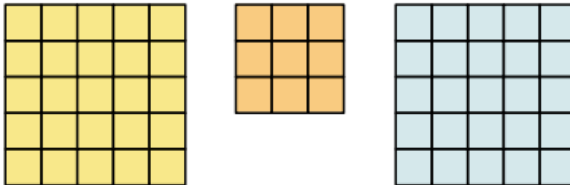
SAME PADDING

Suppose an input of dimensions 5×5 and a filter of size 3×3 .



SAME PADDING

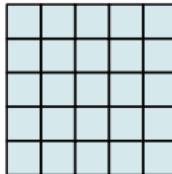
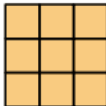
We want an output with the *same* dimensions as the input.



SAME PADDING

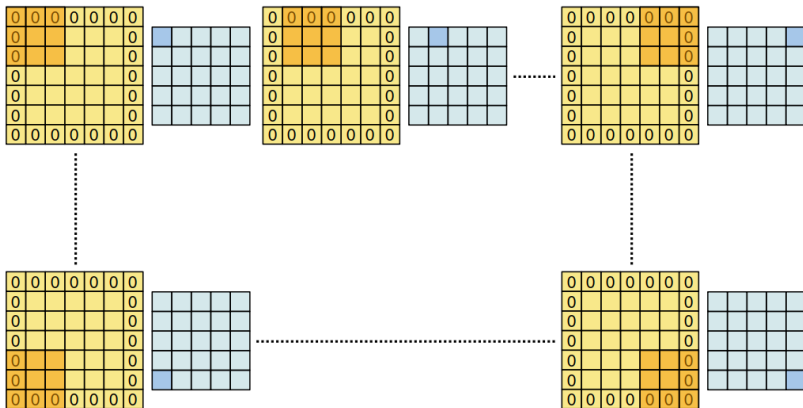
Solution: zero padding — pad zeros around the input.

0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0



SAME PADDING

This always works: we just adjust the amount of padding to the input dimensions and filter size.



PADDING AND NETWORK DEPTH

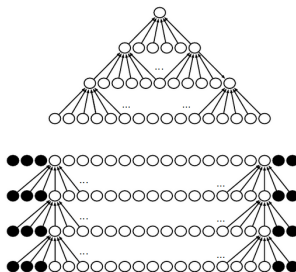


Figure: “Valid” vs. “same” convolution.

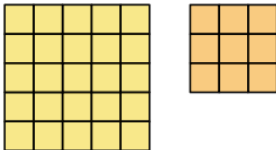
Top: without padding, the feature map width shrinks rapidly (to 1 after just three layers here), limiting network depth.

Bottom: with zero padding, the feature map stays the same size after each convolution, so the network can be made arbitrarily deep (Goodfellow et al., 2016).

Stride

STRIDES

- The step size (“stride”) with which the filter moves (stride = 2 shown below).



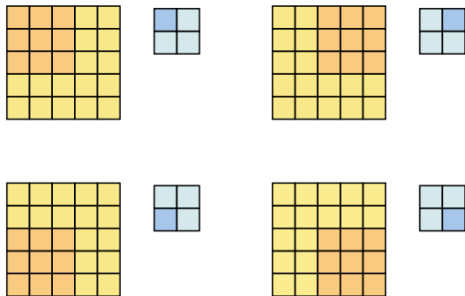
STRIDES

- The step size (“stride”) with which the filter moves (stride = 2 shown below).



STRIDES

- The step size (“stride”) with which the filter moves (stride = 2 shown below).



In general, with no padding, for input size i , filter size k , and stride s , the output size o is:

$$o = \left\lfloor \frac{i - k}{s} \right\rfloor + 1$$

STRIDES AND DOWNSAMPLING

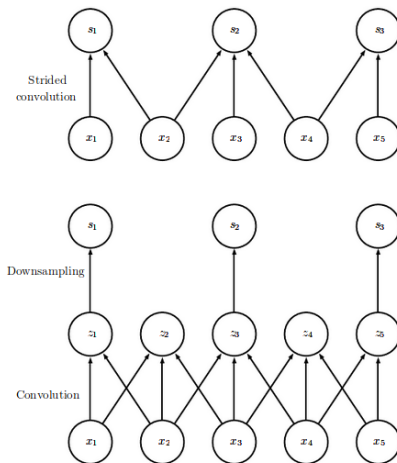


Figure: A strided convolution is equivalent to a convolution without stride, followed by downsampling (Goodfellow et al., 2016).

PUTTING IT TOGETHER

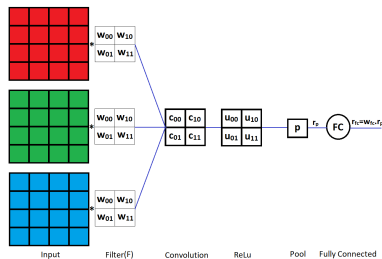


Figure: For color images, Red, Green, and Blue each serve as a separate input channel (Belwal, 2018).

This CNN has:

- 3 input channels, with a 4×4 input matrix each,

PUTTING IT TOGETHER / 2

- one 2×2 filter (kernel),
- a single ReLU layer,
- a single max-pooling layer,
- and a single fully connected (FC) layer.
- The filter entries are equivalent to unit weights in a standard NN, updated during backpropagation.
- With stride 2 and no padding, the output size of the convolution layer is:
- $o = \frac{i-k+2p}{s} + 1$, where i = input size, k = filter size, p = padding, s = stride.

PUTTING IT TOGETHER / 3

Inserting the figure's values:

$$o = \frac{i - k + 2p}{s} + 1 = \frac{(4 - 2 + 2 \cdot 0)}{2} + 1 \quad (1)$$

$$= 2 \quad (2)$$

REFERENCES



Levin, G. (n.d.). *Image Processing and Computer Vision*. ofBook.
https://openframeworks.cc/ofBook/chapters/image_processing_computer_vision.html



Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.



Belwal, C. (2018, June 12). *Part III: Backpropagation mechanics for a convolutional neural network*. <http://cbelwal.blogspot.com/2018/05/part-iii-backpropagation-mechanics-for.html>



Maladkar, K. (2020, October 16). *Computer vision primer: How AI sees an image*. Analytics India Magazine. <https://analyticsindiamag.com/computer-vision-primer-how-ai-sees-an-image/>

Properties of Convolution

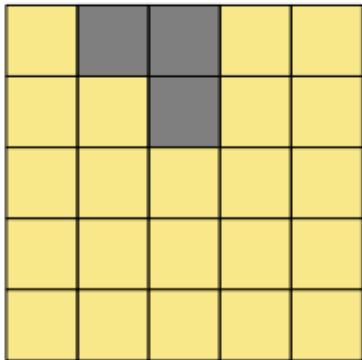
SPARSE INTERACTIONS AND PARAMETER SHARING

- Unlike a dense layer, a convolutional layer connects each output unit only to a small, local **receptive field** of the input — a “local search” for features, instead of a dense layer’s “global search”.
- The same filter weights are reused (**shared**) at every spatial location, instead of learning a separate weight per input–output pair.
- Why this matters: for m inputs and n outputs, a dense layer needs $\mathcal{O}(m \times n)$ parameters and runtime; a convolutional layer with a local filter of size $k \ll m$ needs only $\mathcal{O}(k \times n)$ — drastically less memory, faster runtime, and typically less overfitting.

SPARSE INTERACTIONS AND PARAMETER SHARING / 2

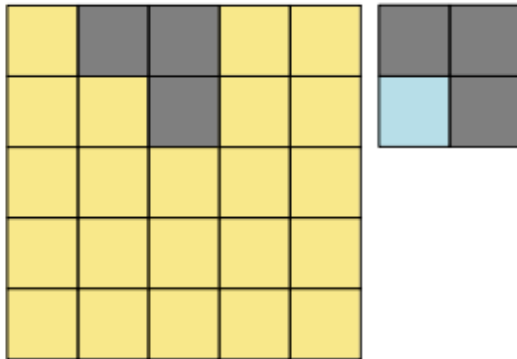
- Example: a 100×100 color image, one same-padded feature map, filter size 5: a conv layer uses $5 \cdot 5 \cdot 3 = 75$ parameters; a dense layer with the same number of hidden units uses $100^2 \cdot 3 \cdot 100^2 = 300,000,000$.

EQUIVARIANCE TO TRANSLATION



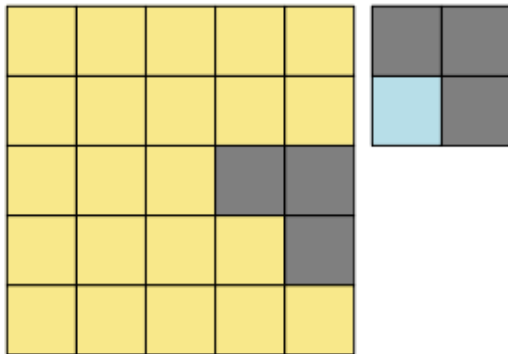
- Think of a specific feature of interest, highlighted in grey.

EQUIVARIANCE TO TRANSLATION



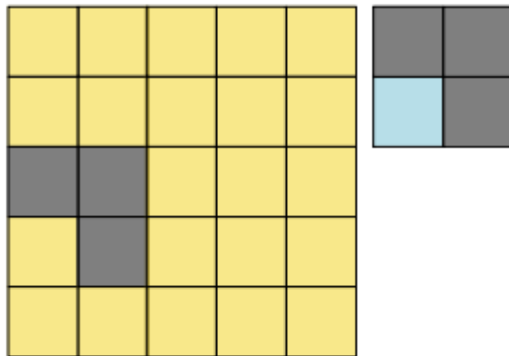
- Assume we have a filter tuned to look for exactly that feature.

EQUIVARIANCE TO TRANSLATION



- The filter does not care *where* the feature is located.

EQUIVARIANCE TO TRANSLATION



- It can find it anywhere — this property is called **equivariance to translation**.

A function f is equivariant to a function g if $f(g(x)) = g(f(x))$.

NONLINEARITY IN FEATURE MAPS

- As in dense nets, we apply an activation function to every feature-map entry to introduce nonlinearity.
- Typically ReLU (or a variant: Leaky ReLU, ELU, PReLU, ..., covered in Session 2) is used:
 - reduces the risk of saturating gradients compared to sigmoid activations,
 - can lead to *sparse activations* (entries ≤ 0 are squashed to 0), which speeds up computation.

Dilated and Transposed Convolutions

DILATED CONVOLUTIONS

- Idea: artificially increase the net's receptive field without adding more filter weights.
- The **receptive field** of a neuron comprises all inputs that affect it. Neurons in early layers capture little of the input; neurons in later layers have huge receptive fields and capture much more global information. Receptive-field size depends on filter size.

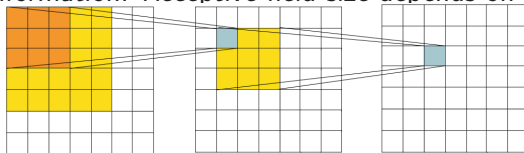
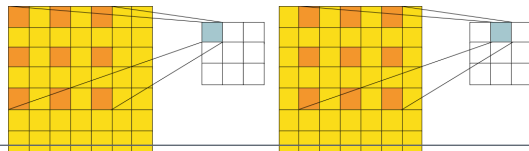


Figure: Receptive field of each layer with a 3×3 kernel. Orange marks the receptive field of one pixel in Layer 2, yellow the receptive field of one pixel in Layer 3.

DILATED CONVOLUTIONS

- Increasing filter size grows the receptive field and captures more context, but also increases the number of parameters and runtime.
- Instead: add a **dilation** parameter that skips pixels during convolution, artificially increasing the receptive field without more weights.
- Benefits: more contextual information, higher-dimensional inputs stay tractable, better runtime (fewer parameters).
- Useful wherever global context matters for the decision — e.g. WaveNet's audio generation, time-series classification/forecasting, image segmentation.



DILATED CONVOLUTIONS

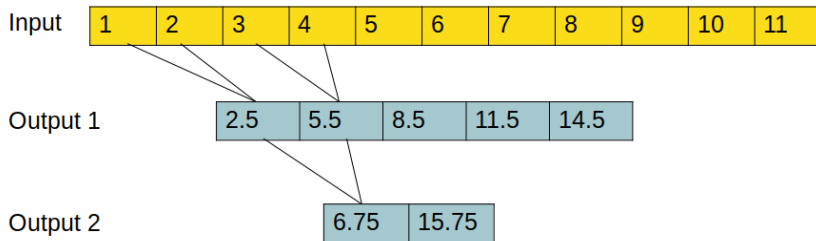


Figure: 1D conv, kernel size 2, stride 2, weights $\{0.5, 1.0\}$, **dilation factor** 1: one neuron in layer 2 has receptive field 4.

DILATED CONVOLUTIONS

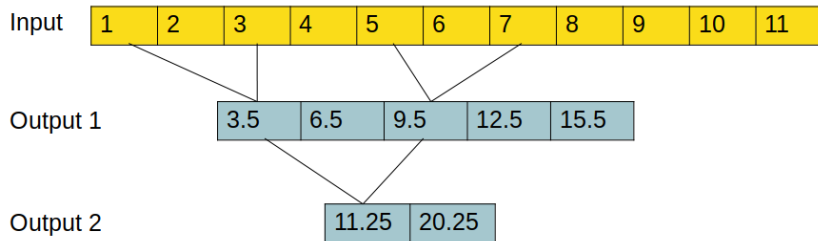


Figure: Same setup with **dilation factor 2**: one neuron in layer 2 now has receptive field 7.

DILATED CONVOLUTIONS

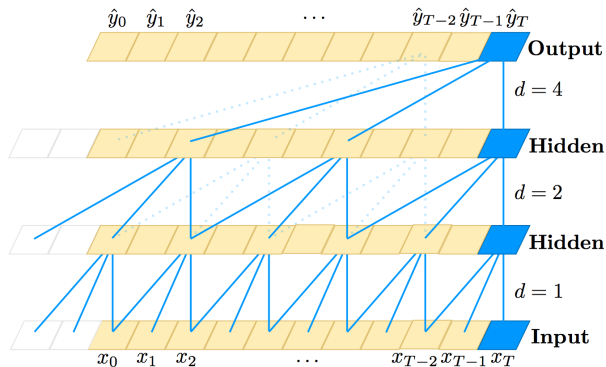


Figure: Dilated convolutions on a time series for classification or seq2seq prediction. Given $x_0, \dots, x_T$, the model outputs $\hat{y}_0, \dots, \hat{y}_T$. Dilation factors $d = 1, 2, 4$ with kernel size $k = 3$ drastically increase context per output neuron with few layers (Roy, 2019).

TRANSPOSED CONVOLUTIONS

- Problem setting: many architectures need to go in the *opposite* direction of a regular convolution, i.e. upsampling — e.g. generating high-resolution images, or mapping a low-dimensional feature map to a high-dimensional space (autoencoders, semantic segmentation).
- Instead of decreasing dimensionality, **transposed convolutions** re-increase it.
- Note: not the same as *deconvolution* (the mathematical inverse of a convolution).

TRANSPOSED CONVOLUTIONS / 2

- Example: input 4×4 (yellow), output 2×2 (blue).

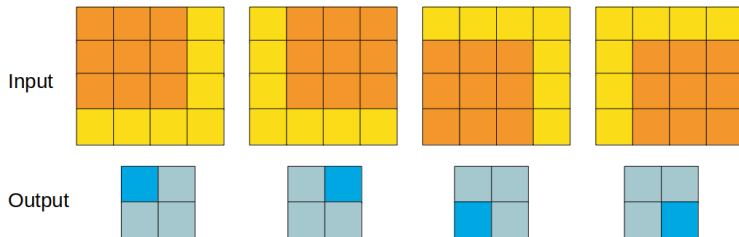


Figure: A **regular** convolution, $k = 3$, $p = 0$, $s = 1$: the feature map shrinks from 4×4 to 2×2 .

TRANPOSED CONVOLUTIONS / 3

- One way to upsample: use a regular convolution with a suitable padding strategy.

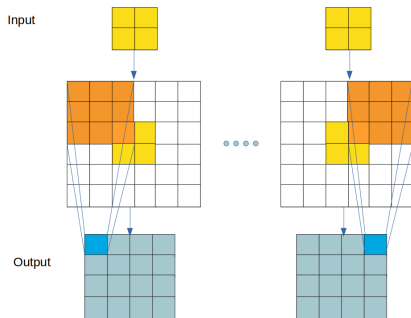


Figure: Transposed convolution as a regular convolution: with $k' = 3, s' = 1, p' = 2$, dimensionality re-increases from 2×2 to 4×4 .

TRANSPOSED CONVOLUTIONS / 4

- In general: a convolution with kernel k , stride s , padding p has an associated transposed convolution with $k' = k$, $s' = s$, $p' = k - 1$.

TRANSPOSED CONVOLUTIONS / 5

Convolution as matrix multiplication:

$$\vec{x} * \vec{a} = X\vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

(Li, 2023)

Figure: A regular 1D convolution, stride 1, padding 1. Vector a is the 1D input feature map.

TRANSPOSED CONVOLUTIONS / 6

Transposed convolution as matrix multiplication:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$
$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

(Li, 2023)

Figure: Upsampling a length-4 vector to length-6, stride 1. Note the change in padding.

TRANSPOSED CONVOLUTIONS / 7

Important: even though the matrix structure here is the transpose of the original, its non-zero elements are, in general, different — these are tuned by backpropagation, not derived analytically.

TRANSPOSED CONVOLUTIONS — A DIFFERENT VIEW

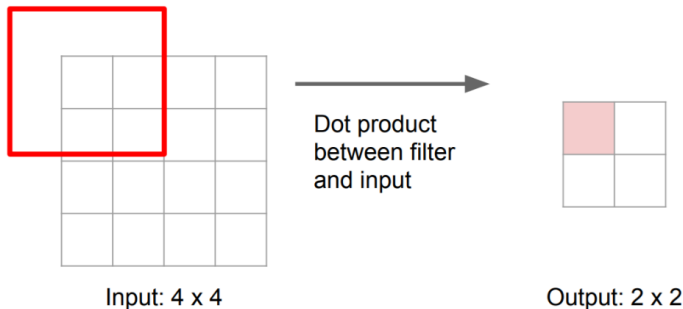


Figure: Regular 3×3 convolution, stride 2, padding 1 (Li, 2023).

TRANSPOSED CONVOLUTIONS — A DIFFERENT VIEW

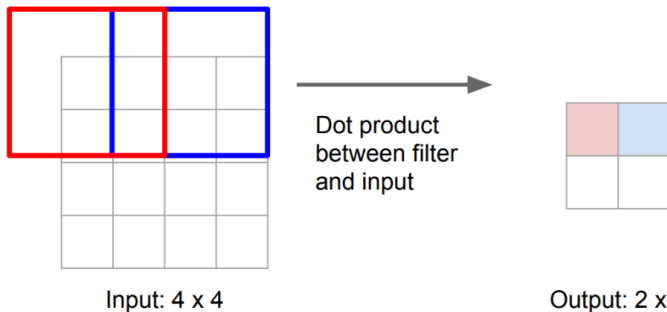


Figure: Regular 3×3 convolution, stride 2, padding 1 (Li, 2023).

TRANSPOSSED CONVOLUTIONS — A DIFFERENT VIEW

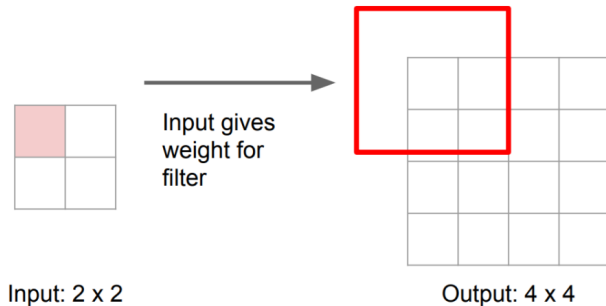


Figure: *Transposed* 3×3 convolution, stride 2, padding 1. Stride now refers to the *output* (Li, 2023).

Here, the filter is *scaled* by the input.

TRANSPOSED CONVOLUTIONS — A DIFFERENT VIEW

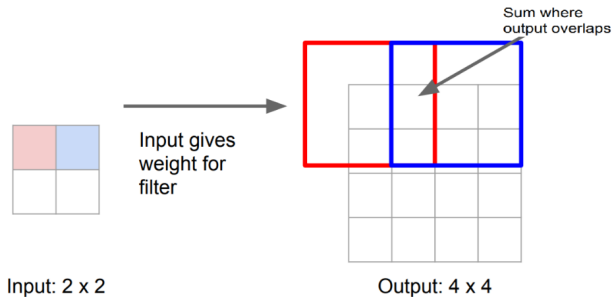


Figure: *Transposed* 3×3 convolution, stride 2, padding 1. Stride now refers to the *output* (Li, 2023).

Here, the filter is *scaled* by the input.

TRANSPPOSED CONVOLUTIONS — A DRAWBACK



Figure: Checkerboard-style artifacts produced by transposed convolutions (Odena et al., 2016).

- Explanation: transposed convolution creates uneven overlap between some feature-map values, giving them higher magnitude than others — the checkerboard pattern.

TRANSPOSED CONVOLUTIONS — A DRAWBACK

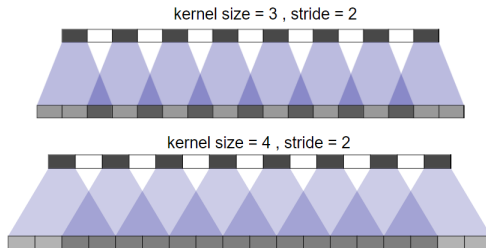


Figure: 1D example (top = input, bottom = output). *Top:* uneven kernel overlap creates a checkerboard pattern. *Bottom:* no checkerboard, because kernel size is divisible by stride (Odena et al., 2016).

- Solutions: upsample first (bilinear / nearest-neighbor) then apply a regular convolution; or ensure the kernel size is divisible by the stride.

TRANSPOSED CONVOLUTIONS — A DRAWBACK

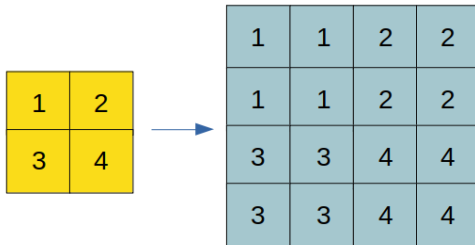


Figure: Nearest-neighbor upsampling followed by a same convolution avoids checkerboard artifacts.

REFERENCES



Roy, R. (2019, February 4). *Temporal Convolutional Networks*. Medium.
[https://medium.com/@raushan2807/
temporal-convolutional-networks-bfea16e6d7d2](https://medium.com/@raushan2807/temporal-convolutional-networks-bfea16e6d7d2)



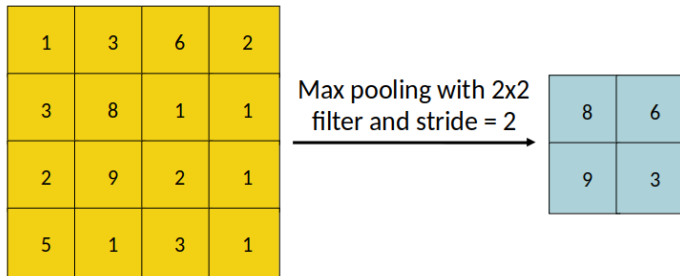
Li, F.-F. (2023). *Lecture 11: Detection and segmentation — Stanford University*. CS231n. [http:
//cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf](http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture11.pdf)



Odena, A., Dumoulin, V., & Olah, C. (2016, October 17). *Deconvolution and checkerboard artifacts*. Distill.
<https://distill.pub/2016/deconv-checkerboard/>

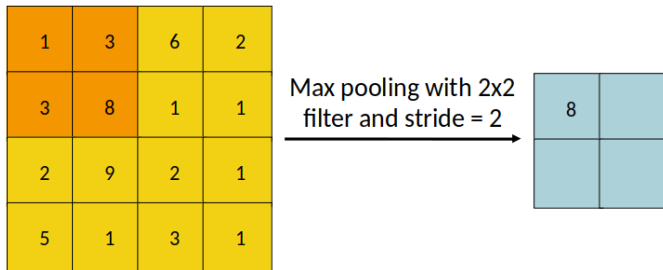
Pooling

MAX POOLING



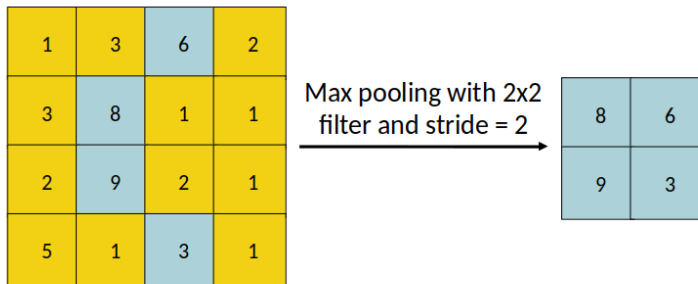
- We've seen how convolutions work — there is one more operation to understand: we want to downsample the feature map while ideally losing no information.

MAX POOLING



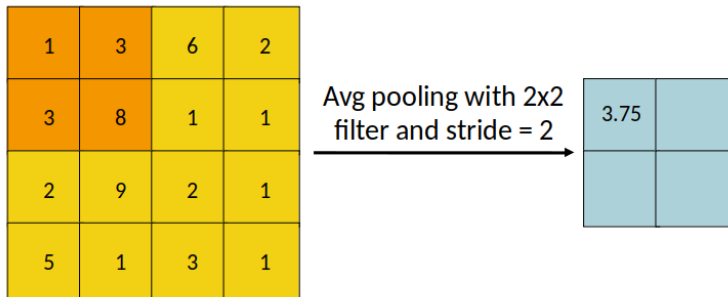
- Max pooling looks for the maximum value at each spatial location — here, 8 for the first location. With a filter of size 2 and no padding, we get a downsampled result.

MAX POOLING



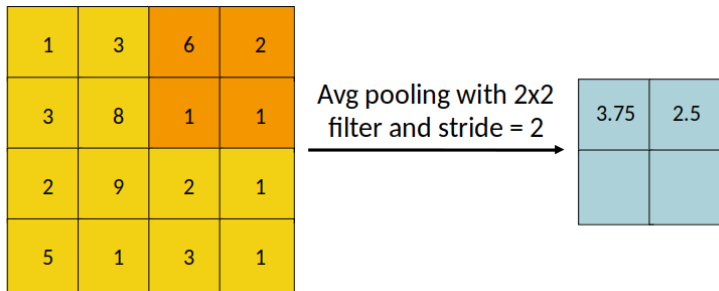
- The final pooled feature map has entries 8, 6, 9, 3. Max pooling brings two properties: dimensionality reduction and (limited) spatial invariance.
- Popular pooling functions: max and (weighted) average.

AVERAGE POOLING



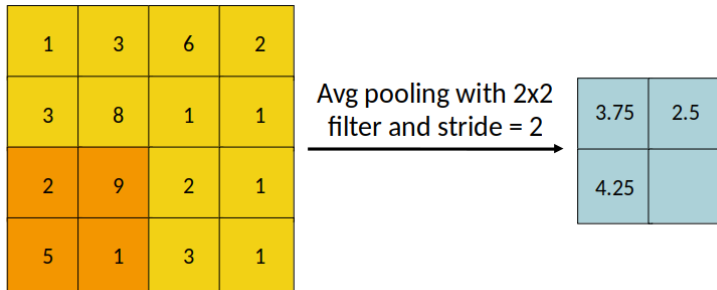
- Other pooling operations exist too: average, fractional, LP, softmax, stochastic, blur, global average pooling, and more. Like max pooling, average pooling downsamples the feature map, but may preserve more information.

AVERAGE POOLING



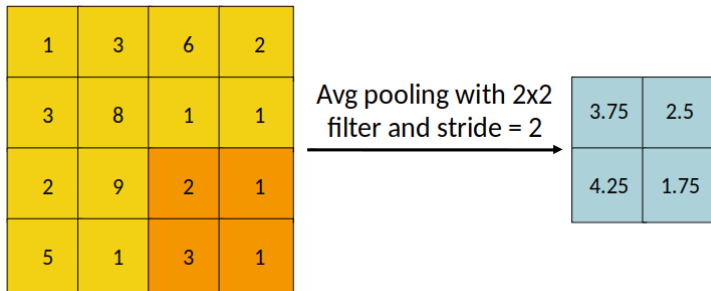
- Average pooling takes the mean value at each spatial location.

AVERAGE POOLING



- It uses all inputs (via the sum) and backpropagates to all of them — but is less robust to noise than max pooling.

AVERAGE POOLING



- The final pooled feature map has entries 3.75, 2.5, 4.25, 1.75.

COMPARISON OF MAX AND AVERAGE POOLING

- Average pooling uses the information of all inputs; max pooling uses only the largest value.
- Max pooling removes detail, so it suits sparse information (typically image classification); average pooling suits dense information (typically NLP).

COMPARISON OF MAX AND AVERAGE POOLING

POOLING / 2

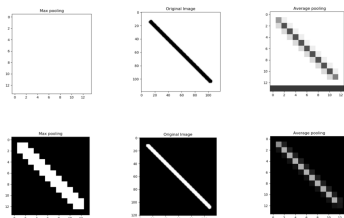


Figure: Shortcomings of max and average pooling on a toy image (Sharma, 2020).

REFERENCES



Sharma, P. (2020, April 21). *MaxPool vs AvgPool*. OpenGenus IQ.
<https://iq.opengenus.org/maxpool-vs-avgpool/>

Modern Architectures

FROM LENET TO MODERN CNNs

A brief tour of how the field evolved, without going deep on any single architecture:

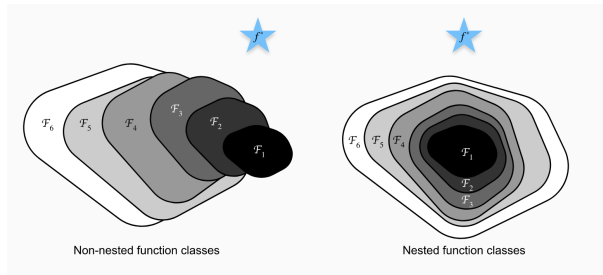
- **LeNet** (LeCun, 1998): the first practical CNN, applied to handwritten-digit recognition (MNIST); depth was mainly limited by 1990s compute.
- **AlexNet** (2012): 8 layers, ReLU instead of sigmoid, won ImageNet (ILSVRC) by a huge margin — the result that triggered the deep learning boom in vision.
- **VGG** (2014): showed that stacking many small 3×3 filters (rather than fewer, larger ones) with more depth improves performance, and introduced the reusable “block” as a network-design unit.

FROM LENET TO MODERN CNNs / 2

- **GoogLeNet / Inception** and **Network-in-Network**: parallel filter paths of different sizes plus 1×1 convolutions to control channel depth cheaply; global average pooling replaces huge dense layers, cutting parameters and overfitting.
- **ResNet** (2015): simply stacking more layers made networks *harder* to train, not easier. The fix — **skip connections**, letting a layer default to the identity function — is important and general enough to cover in depth next.
- **DenseNet**: pushes the skip-connection idea further by concatenating every layer's output into all subsequent layers, not just adding it.
- **U-Net**: a convolutional encoder–decoder using skip connections between mirrored layers — the workhorse of image segmentation.

RESIDUAL NETWORKS: WHY SKIP CONNECTIONS?

Problem setting: in principle, stacking more layers should never hurt — a network could always learn to make extra layers the identity function and skip them.

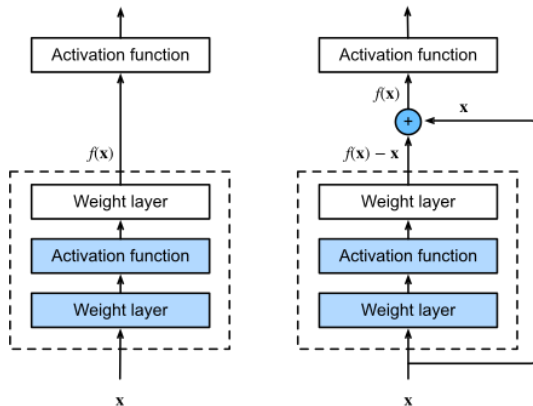


RESIDUAL NETWORKS: WHY SKIP CONNECTIONS? / 2

Figure: For non-nested function classes, a larger function class need not get closer to the true function $\mathcal{F}^*$ — this does not happen for nested function classes (Dive into Deep Learning).

- In practice: learning an identity mapping $x = \mathcal{F}(x)$ through many nonlinear layers is hard, and simply stacking more layers increased *both* train and test error (not overfitting).
- Introduced by He et al. (2015): give the model an explicit shortcut to skip layers that aren't useful.

RESIDUAL NETWORKS: WHY SKIP CONNECTIONS? / 3



RESIDUAL NETWORKS: WHY SKIP CONNECTIONS? / 4

Figure: A regular block (left) vs. a residual block (right): the input is added back via an identity (skip) connection (Dive into Deep Learning).

- Let $\mathcal{H}(x)$ be the mapping a layer should learn. Instead of fitting $\mathcal{H}(x)$ directly, the layer fits the **residual** $\mathcal{F}(x) := \mathcal{H}(x) - x$, while x is added back via the identity: $\mathcal{H}(x) = \mathcal{F}(x) + x$.
- The residual mapping can push weights toward zero to approximate the identity — much easier than learning it directly through nonlinear activations.

RESIDUAL NETWORKS: WHY IT MATTERED

- Inputs and gradients can flow through the identity path even in very deep networks, letting ResNets train reliably at depths (e.g. ResNet-50/101/152) where plain stacked networks degrade.
- ResNet had a major influence on the design of later networks, convolutional and sequential alike.
- Skip connections became a default building block far beyond CNNs — they reappear inside Transformer blocks, covered in Session 7. That's a running theme of this course: a mechanism can outlive the specific architecture it was introduced in, which is exactly why we spend a full session on Transformers next.

REFERENCES



Dive into Deep Learning. (n.d.). 8.6. *Residual Networks (ResNet) and ResNeXt*. http://d2l.ai/chapter_convolutional-modern/resnet.html



He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition*. <https://arxiv.org/abs/1512.03385>

License & Attribution

LICENSE & ATTRIBUTION

- Basic Concepts, The Convolution Operation, 1D/2D/3D Convolutions, CNN Components, Properties of Convolution, Dilated and Transposed Convolutions, and Pooling adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, SLDS, LMU Munich.
https://github.com/slds-lmu/lecture_i2dl — licensed **CC BY 4.0**:
<https://creativecommons.org/licenses/by/4.0/>
- Modern Architectures adapted (and substantially condensed) from the same I2DL source ([slides/cnn3/](#)) — also licensed **CC BY 4.0**.
- Content in this deck has been merged, reordered, and (for Modern Architectures) substantially condensed from the originals; not used verbatim.